

Stratégie de Tests — DataShare

Aperçu

DataShare utilise une approche de test multi-couches combinant des tests unitaires, des tests d'intégration (via Docker Compose) et des tests de bout en bout (E2E) avec Playwright.

Tests Unitaires (Backend — Jest)

Exécution des Tests

```
cd backend
npm test           # Run all tests
npm run test:cov   # Run with coverage report
```

Seuils de Couverture

Métrique	Seuil	Actuel
Statements	70%	72.82%
Branches	50%	80%
Functions	60%	66.66%
Lines	70%	72.31%

La couverture est collectée à partir des fichiers de logique métier (`*.service.ts` , `*.controller.ts` , `*.guard.ts`), en excluant le câblage des modules et les DTOs.

Structure des Tests

Fichier	Tests	Description
auth.service.spec.ts	14	Inscription, connexion, déconnexion, rafraîchissement, génération JWT
auth.controller.spec.ts	4	Points d'entrée du contrôleur (inscription, connexion, déconnexion, rafraîchissement)
jwt.guard.spec.ts	5	Extraction du jeton Bearer, validation, gestion des erreurs
files.service.spec.ts	28	Téléversement, listage, suppression, mot de passe, téléversement anonyme, tags, historique
download.service.spec.ts	13	Création de liens, utilisation des jetons, révocation, expiration
download.controller.spec.ts	4	Points d'entrée du contrôleur (création, listage, révocation, téléchargement)

Patrons de Test

- **Tests de services** : simulent `PrismaService` , `MinioService` et `ConfigService` via des mocks
- **Tests de contrôleurs** : simulent leurs services respectifs + `ConfigService` (pour `JwtGuard`)
- **Tests de garde** : simulent le module `jsonwebtoken` et testent l'extraction/validation des jetons
- Tous les tests utilisent `TestingModule` de `@nestjs/testing` pour une injection de dépendances correcte

Tests E2E (Playwright)

Prérequis

- La pile Docker Compose doit être en cours d'exécution : `make up`
- Le frontend doit être accessible à `http://localhost:3000`

Exécution des Tests E2E

```
cd e2e
npm install
npx playwright test          # Run all E2E tests
npx playwright test --ui     # Interactive mode
npx playwright show-report    # View HTML report
```

Couverture des Tests E2E

Fichier de Test	User Story	Description
us01-upload.spec.ts	US01	Flux de téléversement de fichier
us02-download-links.spec.ts	US02	Génération de lien de téléchargement
us03-register.spec.ts	US03	Inscription d'utilisateur
us04-login.spec.ts	US04	Connexion d'utilisateur
us05-file-list.spec.ts	US05	Listage de fichiers / tableau de bord
us06-stats.spec.ts	US06	Statistiques d'utilisation
us07-password.spec.ts	US07	Protection par mot de passe des fichiers
us08-anonymous-upload.spec.ts	US08	Téléversement anonyme
us09-tags.spec.ts	US09	Étiquetage de fichiers
us10-history.spec.ts	US10	Historique des téléchargements

Patron Page Object

Les tests E2E utilisent le patron Page Object (`e2e/pages/`) pour des sélecteurs maintenables :

- `LoginPage` , `RegisterPage` , `DashboardPage` , `UploadPage`

Intégration CI

Les tests sont prévus pour être exécutés en CI avec :

```
# Unit tests (fast, no dependencies)
cd backend && npm test

# E2E tests (requires Docker stack)
make up
cd e2e && npx playwright test
```

Ajout de Nouveaux Tests

1. **Test de service** : Créer `*.service.spec.ts` à côté du service, simuler les dépendances via `TestingModule`
2. **Test de contrôleur** : Créer `*.controller.spec.ts`, simuler le service + `ConfigService`
3. **Test E2E** : Créer `e2e/tests/usXX-*.spec.ts`, utiliser les Page Objects pour les interactions